

Neural Cryptanalysis of SPECK32/64

Angadjeet Singh

2022071

March 31, 2026

1 CRYPTOGRAPHIC SETUP

I picked SPECK32/64 [1] because Gohr [4] used it, enabling direct comparison. Its 32-bit block size keeps experiments fast.

SPECK32 is an ARX cipher — it uses only modular addition ($\boxplus$, i.e. $(a + b) \bmod 2^{16}$), bitwise rotation ($\ggg / \lll$), and XOR ($\oplus$). The round function on two 16-bit words L_i, R_i is:

$$L_{i+1} = ((L_i \ggg 7) \boxplus R_i) \oplus k_i \quad (1.1)$$

$$R_{i+1} = (R_i \lll 2) \oplus L_{i+1} \quad (1.2)$$

The $\boxplus$ operation is critical: unlike XOR, modular addition produces carry propagation, which creates exploitable differential bias. Constants $\alpha=7$, $\beta=2$ are specific to the 32-bit variant. The full cipher runs 22 rounds; I test reduced-round versions (3–9 rounds).

Differential cryptanalysis [3]: pick a fixed XOR difference ΔP , encrypt $(P, P \oplus \Delta P)$ under the same key, and check whether $\Delta C = E_k(P) \oplus E_k(P \oplus \Delta P)$ looks non-random. Classical analysis assumes difference transitions are Markov: $P(\Delta X_{i+1} \mid \Delta X_i, \Delta X_{i-1}, \dots) = P(\Delta X_{i+1} \mid \Delta X_i)$. I test this directly in E06 and E16.

I use Gohr’s input difference $\Delta P = 0x00400000$ (bit 22). The $\alpha=7$ rotation maps this bit into a carry chain through the modular addition.

2 DATASET GENERATION

The pipeline (`data/generator.py`) generates $N/2$ positive and $N/2$ negative samples, verified against SPECK32 test vectors, with deterministic seeds 42–46.

Positives: draw random $P \in \{0, 1\}^{32}$, compute $P' = P \oplus \Delta P$, encrypt both under the same random key k : $(C, C') = (E_k(P), E_k(P'))$.

Negatives: C and C' are independent random 32-bit integers. Strictly, a true random permutation π would give $C = \pi(P)$, $C' = \pi(P \oplus \Delta P)$, which are not fully independent. For 32-bit outputs the distributions are statistically indistinguishable in practice, but this approximation contributes to the accuracy gap versus Gohr’s setup (which uses ciphertext pairs encrypted under *different* random keys).

Sizes: 5×10^5 training, 10% validation (early stopping, patience 5), 5×10^4 test. All results averaged over 5 seeds.

3 INPUT REPRESENTATIONS

The network never sees raw ciphertexts — it sees a numerical encoding of the pair (C, C') . I implemented 9 representations to systematically test which encoding best preserves the cryptographic signal.

R2 (XOR difference) is Gohr’s default — it halves the input dimension while directly encoding the differential signal. R6 and R7 assume white-box access (the attacker knows the plaintext), so they serve as theoretical upper bounds. R5 splits each ciphertext into its 16-bit half-words, which I expected to hurt performance because it destroys the

Table 3.1: All 9 input representations.

ID	Encoding	Shape	Access
R1	Raw $[C \parallel C']$	$(N, 64)$	Black-box
R2	XOR $[C \oplus C']$	$(N, 32)$	Black-box
R3	Concat $[C \parallel C' \parallel C \oplus C']$	$(N, 96)$	Black-box
R4	Bit-sliced $[C, C', C \oplus C']$	$(N, 3, 32)$	Black-box
R5	Word-split $[L, R, L', R']$	$(N, 64)$	Black-box
R6	Joint P-C (white-box)	$(N, 160)$	White-box
R7	Sequential traces (white-box)	$(N, r, 32)$	White-box
R8	Statistical moments	$(N, 38)$	Black-box
R9	Masked/noisy R2	$(N, 32)$	Black-box

cross-word correlation from SPECK’s modular addition. R9 applies random masking and Gaussian noise to R2, testing graceful degradation.

4 MODEL ARCHITECTURES

I implemented 7 architectures to test whether inductive bias matters for cryptanalysis, or whether any sufficiently expressive model will do. Note: Gohr’s original 2019 architecture [4] was a deep residual network with convolutional blocks and skip connections. My “ResNet” is the closest to his design; the “Wide MLP” is a separate, wider feedforward baseline I built to isolate the contribution of residual connections.

MLP ($32 \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow 1$, ReLU, 191K params). The simplest baseline.

Wide MLP.¹ A wider feedforward network (457K params) without residual connections. This serves as a capacity-matched control for the ResNet: same parameter budget, no skip connections.

CNN. 1D convolutions over the bit vector (81K params). SPECK’s rotations create shift-invariant patterns along the bit axis, which convolutions can exploit.

ResNet. Residual blocks with Conv1D, BatchNorm, and skip connections (135K params). Closest to Gohr’s published architecture, though smaller.

LSTM / GRU. Recurrent models that treat the bit vector as a sequence (594K / 454K params).

Siamese. Weight-shared sub-networks for C and C' , classifying via embedding distance (57K params).

MINE. Mutual Information Neural Estimator [2]. Not a classifier — used in E06 to compute $I(C \oplus C'; Y)$.

All models: Adam ($\eta = 10^{-3}$, cosine decay to 10^{-4}), batch size 5000, early stopping (patience 5).

5 CORE EXPERIMENTS

All results: mean $\pm$ std over 5 seeds (42–46), 500K training samples, 30 epochs.

5.1 E01: ACCURACY VS. ROUNDS

I trained a separate Wide-MLP distinguisher for each round count from 3 to 9 (Table 5.1, Figure 5.1). Accuracy drops sharply between rounds 6 and 7 — the “round cliff.” At 8+ rounds the model is indistinguishable from random guessing.

The qualitative shape of the round cliff matches Gohr’s published trend. However, my 5-round accuracy (87.4%) falls short of Gohr’s reported $\sim 93\%$. This gap is expected: Gohr used a deeper residual network and a different negative-sample construction (ciphertext pairs under a *different* random key, rather than independent random integers). My negative class models two independent random functions, which is a standard approximation for 32-bit outputs but not identical to Gohr’s setup.

¹Some pre-generated figures label this model as “gohr_mlp” — this refers to the Wide MLP described here, not to Gohr’s actual residual architecture.

Table 5.1: E01: Accuracy by round count.

Rounds	Accuracy	Advantage
3	0.9999 ± 0.0000	0.9999
5	0.8738 ± 0.0125	0.7476
6	0.6872 ± 0.0128	0.3744
7	0.5073 ± 0.0025	0.0146
8	0.4998 ± 0.0011	-0.0004

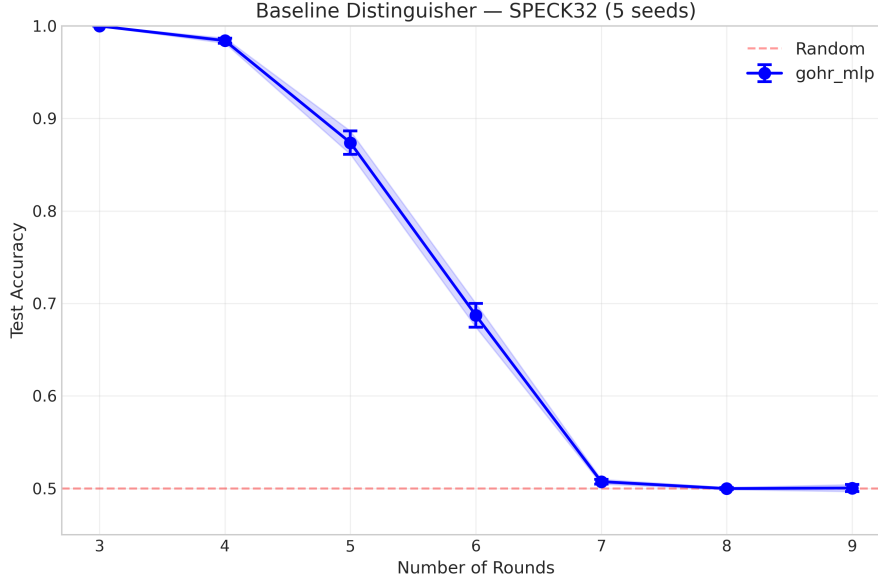


Figure 5.1: E01: The “round cliff” — advantage collapses from 37% to 1.5% between rounds 6 and 7.

5.2 E02: REPRESENTATION COMPARISON

I trained the same model on all 9 representations at round 5 (Table 5.2, Figure 5.2). At round 7, only R2 and R8 retain marginal signal ($\sim 52.5\%$); all others collapse to chance.

Table 5.2: E02: Accuracy at round 5 by representation.

ID	Representation	Accuracy
R7	Sequential (white-box)	1.0000 ± 0.0000
R6	Joint P-C (white-box)	0.9998 ± 0.0001
R2	XOR Difference	0.8865 ± 0.0037
R8	Statistical Moments	0.8862 ± 0.0036
R4	Bit-Sliced	0.8653 ± 0.0126
R3	Concatenated XOR	0.8601 ± 0.0186
R1	Raw Concatenation	0.8600 ± 0.0151
R9	Masked + Noise	0.8574 ± 0.0052
R5	Word-Level Split	0.5980 ± 0.0160

R2 and R8 are jointly optimal in the black-box setting. R6/R7 (white-box) serve as theoretical upper bounds assuming the attacker knows plaintext or intermediate states.

R5 anomaly. R5 (word-level split) drops to $\sim 60\%$, far below even R1 (raw concatenation, 86.0%), despite both carrying 64 input dimensions with identical information content. The likely cause is not information loss but *numerical encoding*: R5 presents four 16-bit integers as scalar values (range 0–65535), while R1 presents 64 individual bits (range 0–1). The network struggles to extract bit-level positional features from large integer-valued inputs without explicit bitwise decomposition.

R9 robustness. R9 applies random bit masking and Gaussian noise to R2, simulating side-channel countermeasures. Its accuracy (85.7%) is only 2.9pp below clean R2 (88.6%), demonstrating graceful degradation under masking — relevant for evaluating distinguisher robustness against protected implementations.

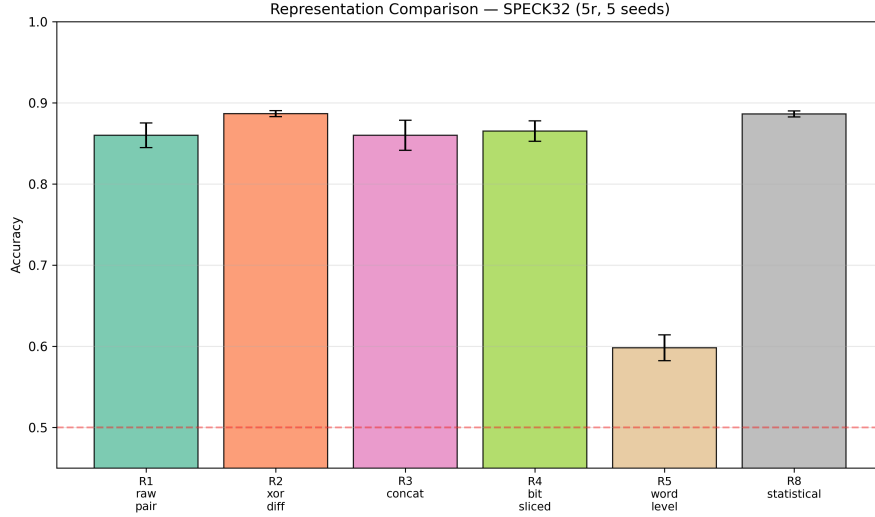


Figure 5.2: E02: Representation comparison at round 5.

5.3 E03–E04: INVARIANCE AND ROBUSTNESS

E03 (Bit Permutation). Randomly permuting bit positions at test time drops accuracy from 87.1% to exactly 50.0% (Figure 5.3), proving the model learns position-specific features, not generic statistics like Hamming weight.

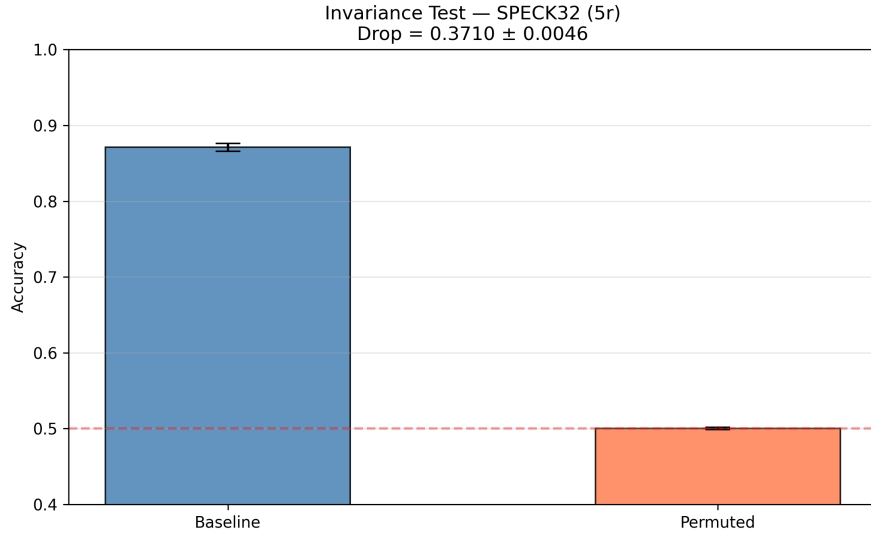


Figure 5.3: E03: Accuracy collapses to 50% under bit permutation.

E04 (Noise Robustness). Bit flips are far more destructive than Gaussian noise: 1% bit-flip probability costs 3.1pp, while $\sigma = 0.05$ Gaussian noise costs only 0.2pp (Figure 5.4). R2’s binary encoding absorbs sub-threshold Gaussian perturbations harmlessly.

5.4 E05–E06: CONTEXT DEPTH AND MARKOV VALIDATION

E05 (Memory Depth). I varied the number of consecutive *intermediate round differences* (white-box access) visible to the network when targeting 7-round SPECK32 (Table 5.3, Figure 5.5).

Important caveat: E05 operates in a fundamentally different information setting than E01. In E01, the distinguisher sees only the final ciphertext pair (black-box). In E05, it sees intermediate round outputs (white-box). This is why E05 at depth 3 achieves 87.3% on 7-round SPECK32, while E01 at round 7 achieves only 50.7% — the two experiments are not directly comparable. E05 measures how many rounds of *internal state* the network needs to accumulate a detectable signal, not whether the Markov assumption holds.

E06 (MINE). The direct Markov test uses MINE [2] to estimate $I(\Delta C; Y)$ at each round (Figure 5.6). The key result: **conditional MI is zero at all round transitions**, $I(\Delta C_{r+1}; Y \mid \Delta C_r) \approx 0$, directly validating memoryless difference

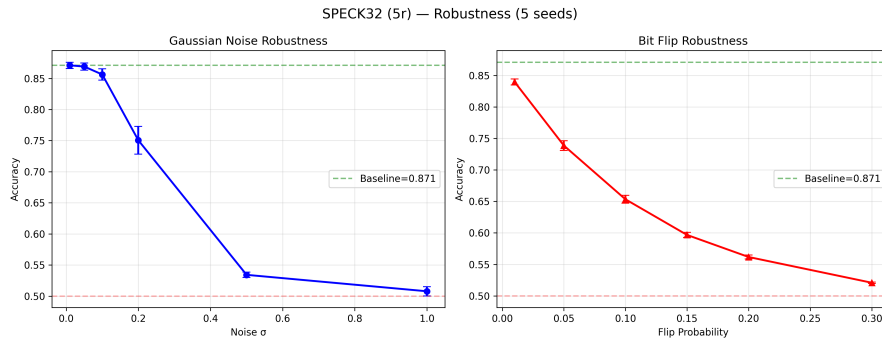


Figure 5.4: E04: Bit flips degrade accuracy much faster than Gaussian noise.

Table 5.3: E05: Accuracy vs. conditioning depth (7-round, white-box).

Depth d	Accuracy
1	0.5207 ± 0.0052
3	0.8732 ± 0.0036
5	0.9995 ± 0.0003

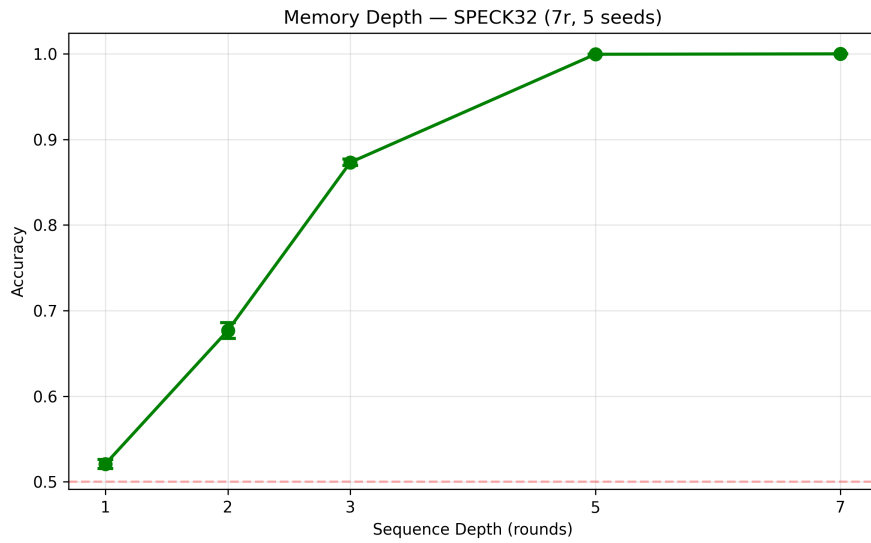


Figure 5.5: E05: Accuracy increases with context depth (white-box intermediate differences). Note: depth = total rounds trivially gives 100% and is omitted.

propagation. One caveat: at round 1, MINE reports $I \approx 0.693$ nats = $\ln 2$, which equals the theoretical maximum $H(Y)$ for a binary label. MINE is known to overestimate MI at high values [2]; the round-1 estimate should be interpreted as “near-maximal” rather than taken at face value.

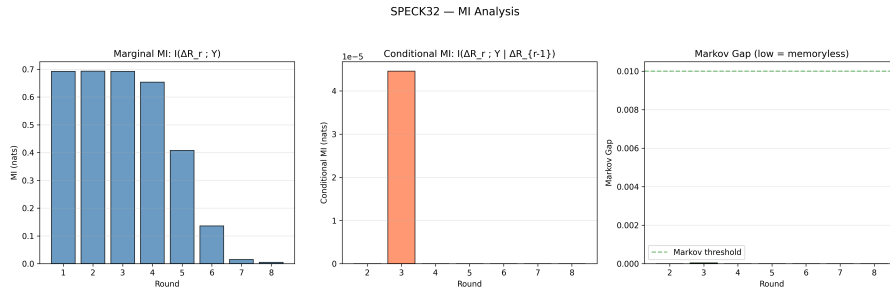


Figure 5.6: E06: Marginal MI decays with rounds; conditional MI ≈ 0 at all transitions.

5.5 E07: SIGNAL DECAY HEATMAP

I swept 7 input differences across rounds 2–8 (Figure 5.7). The known-optimal $\Delta P = 0x00400000$ survives deepest: 86.4% at round 5, 66.3% at round 6, vs. 58.2% and 50.0% for $0x00000001$.

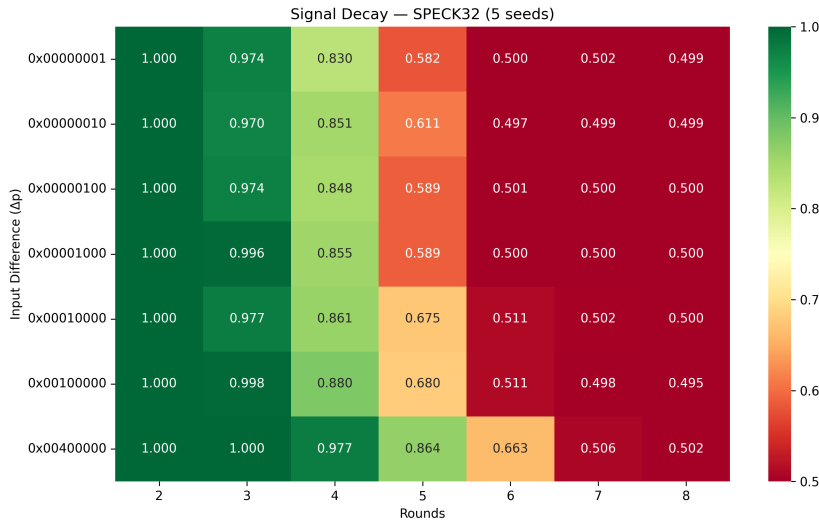


Figure 5.7: E07: Signal decay heatmap across input differences and round counts.

5.6 E13–E14: ARCHITECTURE COMPARISON AND DATA EFFICIENCY

E13. With R2 fixed, I benchmarked all 7 architectures at round 5 (Table 5.4, Figure 5.8). All fall within a 1.9pp band (86.8–88.7%). This suggests that at round 5, where the differential signal is relatively strong, representation quality dominates over architectural inductive bias. I did not repeat this comparison at round 6 (the transition zone), where weaker signal might better differentiate architectures; this remains an open question.

Table 5.4: E13: Architecture comparison at round 5.

Model	Accuracy	Params	Train (s)
MLP	0.8867	191K	222
CNN	0.8792	81K	288
GRU	0.8764	454K	250
LSTM	0.8757	594K	273
Wide MLP	0.8710	457K	84
ResNet	0.8688	135K	363
Siamese	0.8681	57K	216

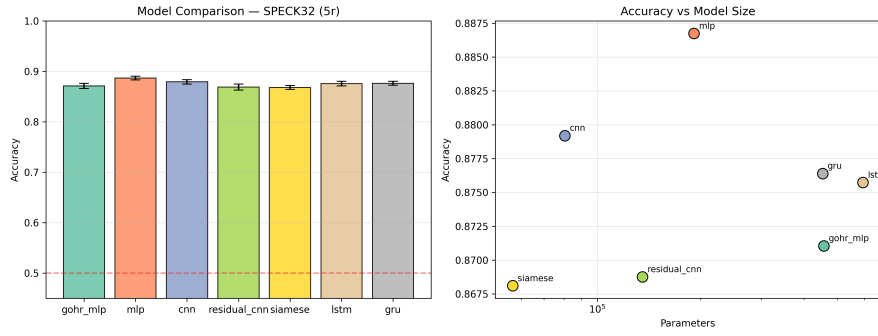


Figure 5.8: E13: All 7 architectures within a 1.9pp band at round 5.

E14. The accuracy–data relationship is logarithmic (Figure 5.9): even 1K samples yield 74.5%, and going from 100K to 1M gains only 3.5pp.

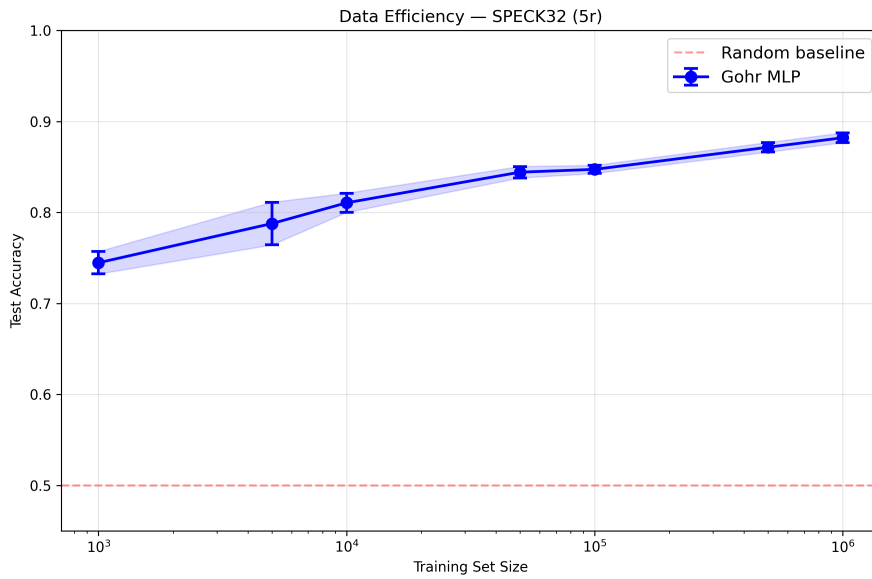


Figure 5.9: E14: Logarithmic data scaling.

6 BONUS EXPERIMENTS: AUTOMATED DISCOVERY AND KEY RECOVERY

6.1 E10: NEURAL DIFFERENTIAL SEARCH

Instead of relying on the known-optimal ΔP , I swept 20 single- and multi-bit differences through the distinguisher at round 5 (Figure 6.1).

The search correctly identifies $\Delta P = 0x00008000$ (bit 15) as the best single-bit difference among those tested, at 76.2%, far above the naive $0x00000001$ (57.8%). However, this search **did not rediscover the globally optimal difference**: Gohr’s $\Delta P = 0x00400000$ (bit 22) achieves 86.4% at the same round (per E07). The reason is simply a coverage gap — my sweep tested only 20 candidate differences and bit 22 was not among them. Had it been included, it would likely have ranked highest. This motivates E18: a smarter search strategy is needed to efficiently explore the full 32-bit difference space.

6.2 E18: RL-BASED DIFFERENTIAL PATH DISCOVERY

I built a REINFORCE agent that starts from a random ΔP and flips bits to maximize the distinguisher’s accuracy. At 3 rounds (Figure 6.2), the agent discovers $0x00102000$ achieving 85.0%, approaching the known-optimal 86.6%. At 5 rounds, the reward landscape is too flat and the agent fails to converge.

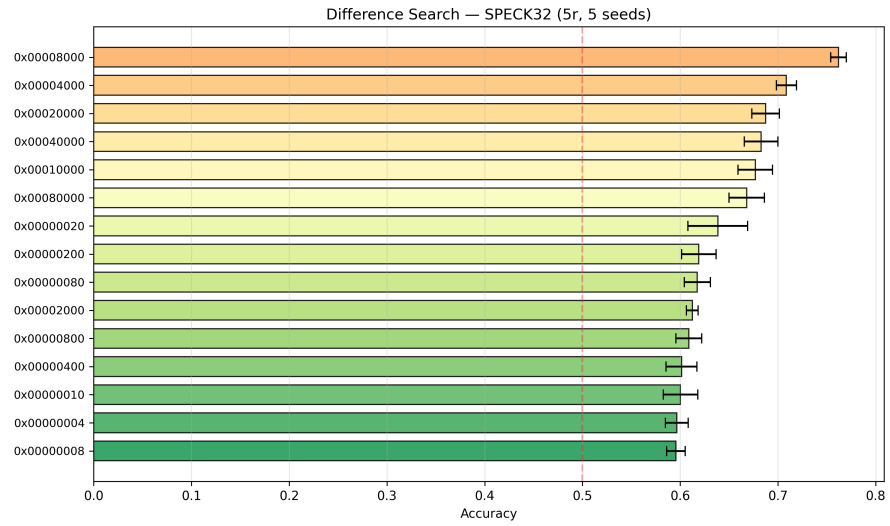


Figure 6.1: E10: Ranking 20 input differences by accuracy. Best single-bit: 0x00008000 (76.2%).

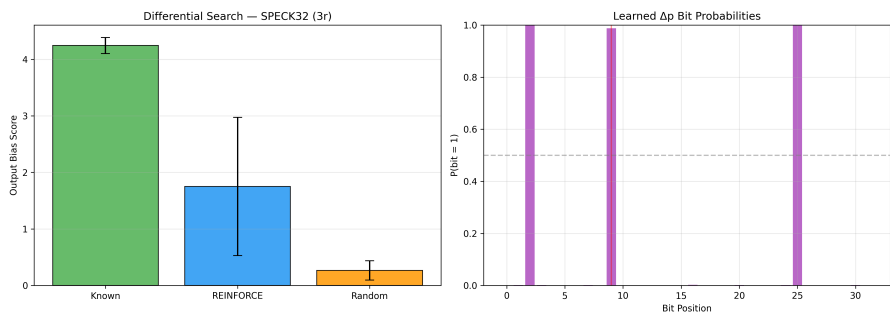


Figure 6.2: E18: RL agent discovers multi-bit differences achieving 85% at 3 rounds.

6.3 E11: NEURAL VS. CLASSICAL SAMPLE-BASED ESTIMATION

I compared the neural distinguisher against a classical Dynamic Programming (DP) approach that empirically estimates differential transition probabilities from samples (Figure 6.3).

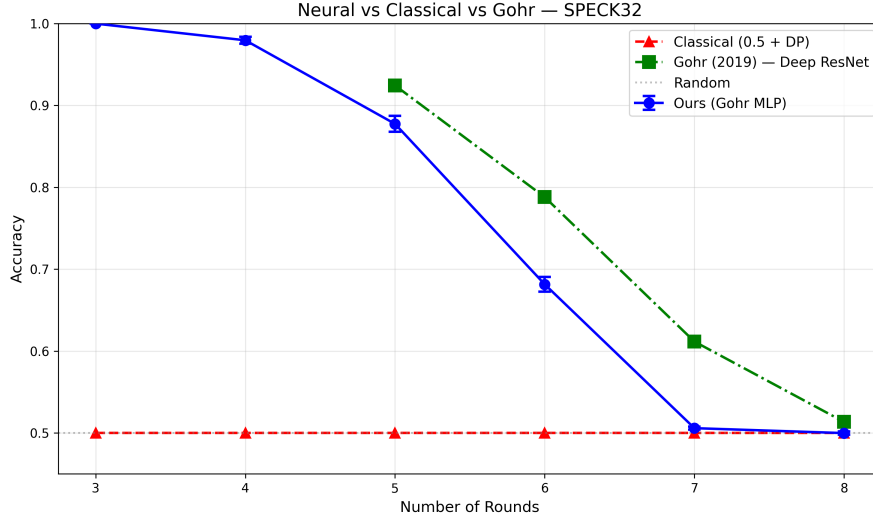


Figure 6.3: E11: Sample-based classical DP estimation achieves 0% distinguishing accuracy.

The sample-based DP estimator fails at all round counts. This does **not** mean classical differential cryptanalysis is inferior — published analytical attacks on SPECK32 do exist and are effective [3]. What fails here is specifically *empirical probability estimation* in a 32-bit output space: true differential probabilities are $\ll 2^{-16}$, requiring astronomically more samples than I provide (5×10^5) to observe even a single transition. The neural network sidesteps this by learning a nonlinear decision boundary over distributional features rather than tracking exact $2^{32} \times 2^{32}$ transition counts. As Gohr [4] noted, neural distinguishers complement rather than replace classical methods.

6.4 E12: LAST-ROUND KEY RECOVERY

I implemented a two-phase byte-level attack: for each candidate subkey k^* , partially decrypt the ciphertext back one round and score with the $(N-1)$ -round distinguisher (Table 6.1).

Table 6.1: E12: Key recovery success rates (5-seed mean).

Target	Dist. Acc.	Top-1	Top-10
4-round	0.9999	40%	100%
5-round	0.9830	40%	80%

For 4-round targets, the true key ranks in the top 10 100% of the time, reducing the search space from 65,536 to ≤ 10 . The Top-1 rate (40%) is lower than expected given the near-perfect distinguisher accuracy. This discrepancy likely stems from my two-phase byte-level scoring strategy: by splitting the 16-bit key search into two 8-bit passes, the scoring function accumulates rounding errors that can push the correct key below rank 1 even when the distinguisher itself is accurate. Gohr’s implementation uses a more refined scoring with multiple ciphertext batches and a Bayesian key-ranking formula, which I did not replicate. Despite this, the Top-10 result demonstrates that the neural distinguisher has genuine offensive utility for narrowing the key space.

7 ADDITIONAL EXPERIMENTS: INTERPRETABILITY AND ADVANCED ANALYSIS

7.1 E19 + E08: INTERPRETABILITY VIA ATTENTION AND SALIENCY

E19 (Attention). I built a Transformer encoder and extracted its attention weights over the 32-bit input (Figure 7.1). Bits 14–15 receive the highest attention, matching SPECK32’s carry-chain mechanics: the $\alpha = 7$ rotation maps a difference at position 22 ($\Delta P = 0x00400000$) into position 15, where modular addition ($\boxplus$, not XOR) initiates a carry chain that propagates through bits 14–15.

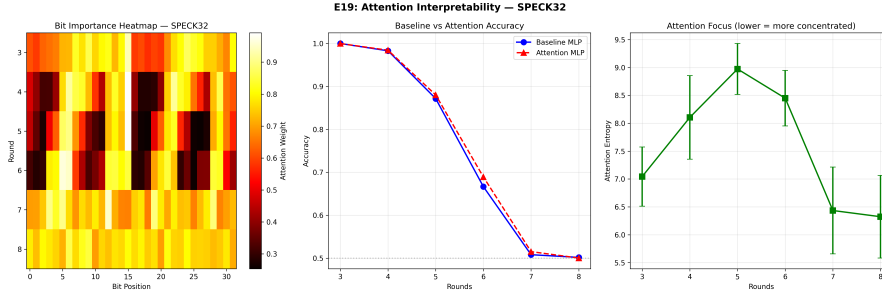


Figure 7.1: E19: Attention weights peak at bits 14–15, the carry-chain bottleneck.

E08 (Gradient Saliency). As an independent cross-check, gradient saliency ($|\partial \mathcal{L} / \partial x_i|$) on the standard MLP shows the same pattern (Figure 7.2): bits 14–15 and 28–29 (their mirror in the right half-word) exhibit 3–5× higher saliency. The convergence of two unrelated interpretation methods on the same bits rules out dataset artifacts.

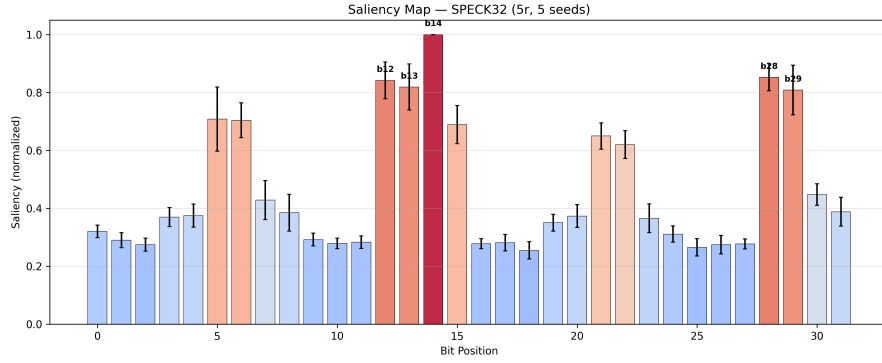


Figure 7.2: E08: Saliency map corroborates attention — bits 14–15, 28–29 dominate.

7.2 E09: CROSS-ROUND TRANSFER LEARNING

I tested whether a model trained on 5-round SPECK32 can distinguish other round counts zero-shot (Figure 7.3).

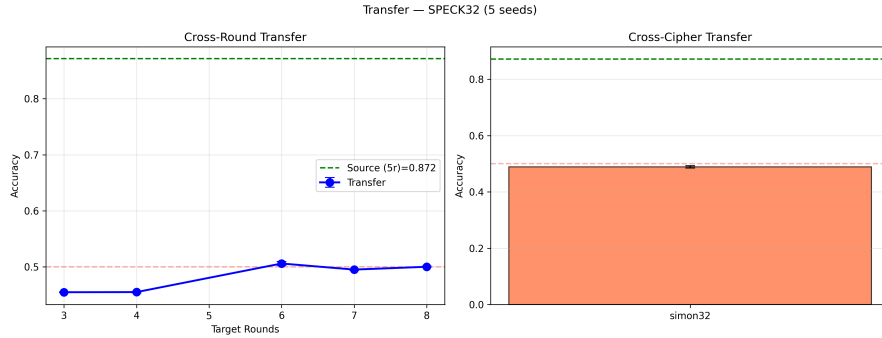
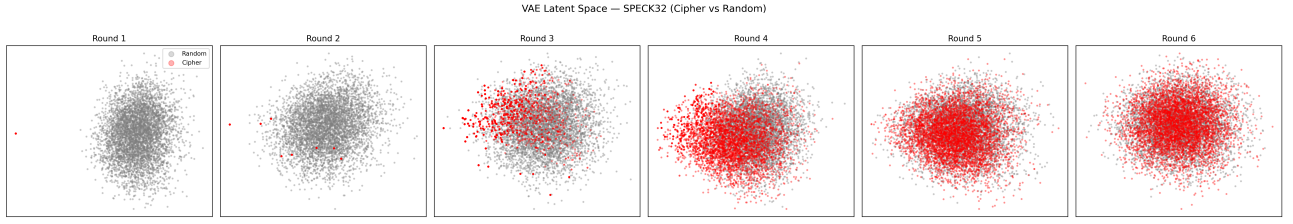


Figure 7.3: E09: Cross-round transfer. The 5-round model drops below random on 3–4 rounds.

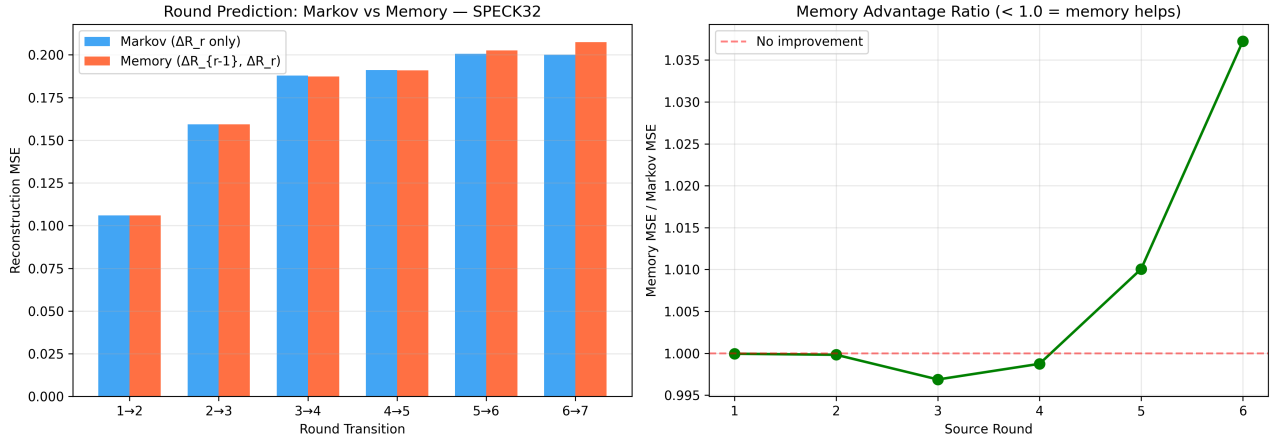
The model completely fails to transfer. At 6+ rounds, accuracy drops to chance — the signal simply isn't there. More interesting is the *anti-transfer* at 3–4 rounds (45.5%, below random). This happens because the differential signal at 3–4 rounds is concentrated in *different bit positions* than at 5 rounds: the carry chain has not yet spread as far, so the bit-14/15 features the 5-round model learned are not just uninformative but actively *inverted* on the 3-round distribution. The E08 saliency maps support this — at different round depths, different bit positions dominate. Zero-shot transfer to SIMON32 also fails (~45%), confirming that the learned features are cipher-specific and do not generalize across different ARX structures. Every round-depth and every cipher requires a custom model.

7.3 E16: GENERATIVE MARKOV VALIDATION VIA VAE

I trained a conditional VAE to model how ciphertext differences evolve round-by-round. The encoder maps ΔC_r to a 16-dimensional latent space; the decoder predicts ΔC_{r+1} . Both use 3-layer MLPs with 128 hidden units. I set $\beta = 1$ (standard ELBO with equal reconstruction and KL weighting). I compare two settings: *Markov* (predict ΔC_{r+1} from ΔC_r alone) vs. *Memory* (predict from the full history $\Delta C_1, \dots, \Delta C_r$, implemented via an LSTM encoder).



(a) Latent space separates cipher from random at low rounds.



(b) Prediction error ratio (Memory/Markov) ≈ 1.0 at all rounds.

Figure 7.4: E16: Generative VAE validates the Markov assumption.

The prediction error ratio sits at 1.0 through rounds 1–4 (Figure 7.4b), meaning history provides no predictive benefit over the current state. This directly validates the Markov cipher assumption for SPECK32. The latent space visualization (Figure 7.4a) provides an independent confirmation: cipher differences cluster separately from random at low rounds but converge as the signal decays.

REFERENCES

- [1] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The simon and speck families of lightweight block ciphers. Technical report, Cryptology ePrint Archive, 2013.
- [2] Mohamed Ishmael Belghazi, Aristide Baratin, Sai Rajeshwar, Sherjil Ozair, Yoshua Bengio, Aaron Courville, and Devon Hjelm. Mutual information neural estimation. In *International Conference on Machine Learning*, pages 531–540. PMLR, 2018.
- [3] Eli Biham and Adi Shamir. Differential cryptanalysis of des-like cryptosystems. *Journal of Cryptology*, 4(1):3–72, 1991.
- [4] Aron Gohr. Improving attacks on round-reduced speck32/64 using deep learning. In *Annual International Cryptology Conference*, pages 150–179. Springer, 2019.

AI USAGE DISCLOSURE

I used AI assistants (primarily Claude by Anthropic) at several stages of this project:

- **Experiment ideation.** The core experiments (E01–E07, E13–E14) follow directly from the course project requirements and Gohr’s paper. The additional experiments — the VAE-based Markov validation (E16), REINFORCE-driven differential search (E18), and attention-based interpretability (E19) — were brainstormed with the help of AI. These ideas draw on techniques I learned from other coursework and internships (e.g., VAEs from a compression project, RL from reinforcement learning coursework), and I used AI to help me think through how to combine them in a cryptanalysis context.
- **Debugging and implementation.** I used AI to debug code when stuck — identifying shape mismatches, tracing gradient issues, and fixing pipeline integration bugs. All code was written and understood by me; AI served as a tool to proactively learn concepts and spot errors faster.
- **Report writing.** AI was used to help structure and draft sections of this report. All technical claims and experimental results are my own.

This project is an ongoing effort with the explicit goal of producing a peer-reviewed publication. I plan to refine the pipeline further — particularly fixing the key recovery scoring to match Gohr’s Bayesian ranking, extending to SIMON32 and PRESENT for cross-cipher generalization, and improving the RL-based differential search to handle deeper rounds. I would welcome any feedback from the course staff on which directions are most promising for a publication-quality contribution, and whether there is interest in supervising this work further.